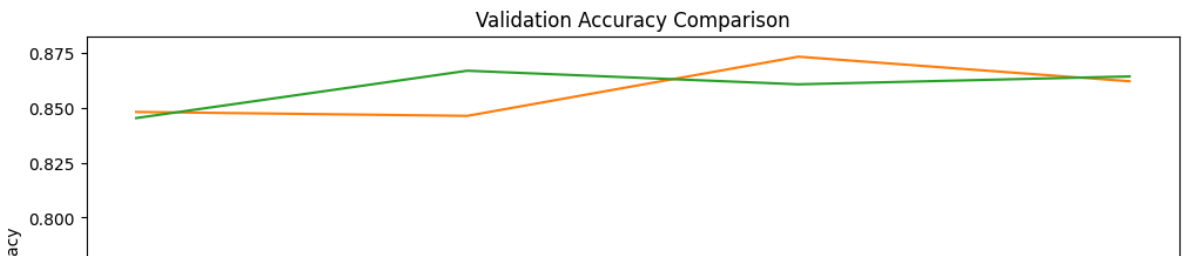
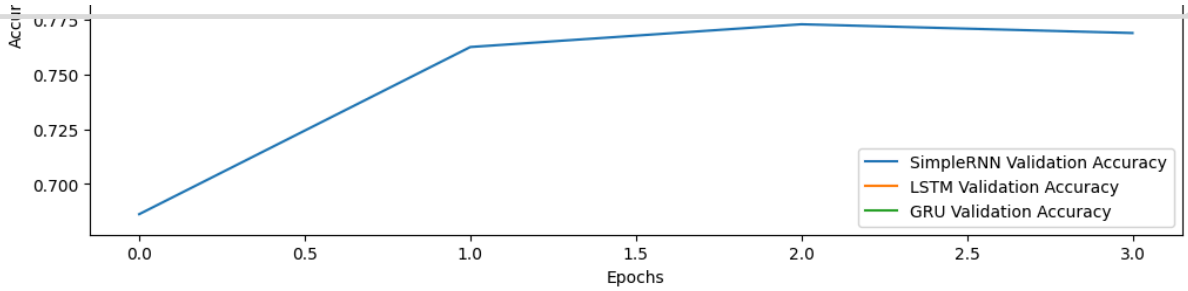
Assignment 7

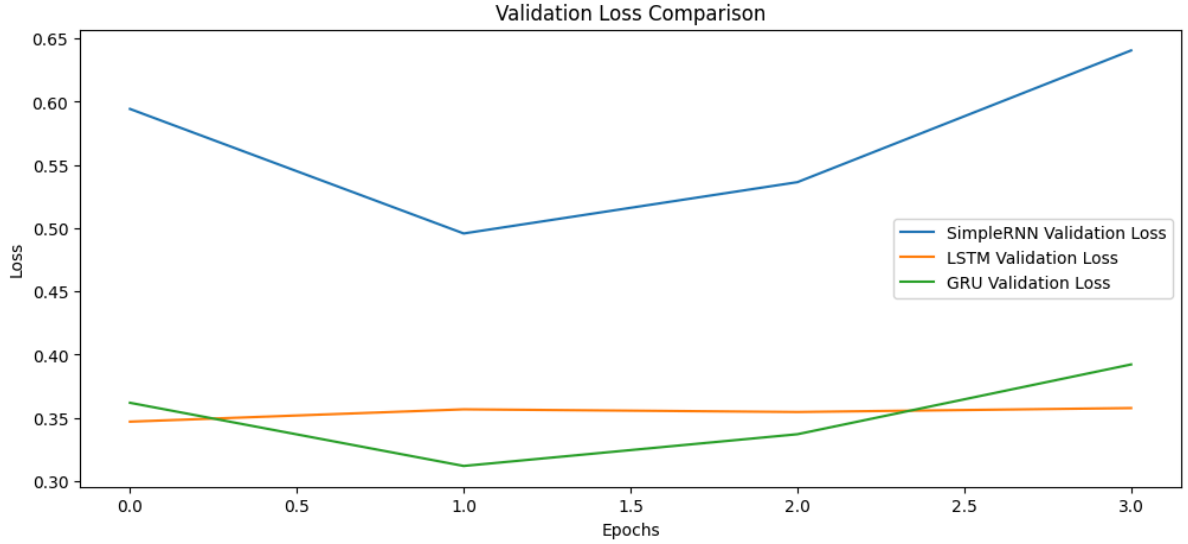
```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import imdb
from tensorflow.keras.preprocessing import sequence
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, LSTM, GRU
from tensorflow.keras.optimizers import Adam

# Step 2️⃣ : Load and Preprocess Data
max_features = 10000   # Use top 10,000 words
maxlen = 200           # Cut reviews after 200 words

print("Loading IMDB dataset...")
(x_train, y_train), (x_test, y_test) = imdb.load_data(num_words=max

print(f"Training samples: {len(x_train)}, Test samples: {len(x_test

# Pad sequences to ensure equal length
x_train = sequence.pad_sequences(x_train, maxlen=maxlen)
x_test = sequence.pad_sequences(x_test, maxlen=maxlen)

print("Data shape after padding:", x_train.shape, x_test.shape)

# Step 3️⃣ : Build Models (SimpleRNN, LSTM, GRU)
def build_rnn(model_type="SimpleRNN", activation="tanh"):
    model = Sequential()
    model.add(Embedding(max_features, 128, input_length=maxlen))
    if model_type == "SimpleRNN":
        model.add(SimpleRNN(64, activation=activation))
    elif model_type == "LSTM":
        model.add(LSTM(64, activation=activation))
    elif model_type == "GRU":
        model.add(GRU(64, activation=activation))
    model.add(Dropout(0.5))
    model.add(Dense(1, activation='sigmoid'))

    model.compile(loss='binary_crossentropy', optimizer=Adam(0.001)
    return model

# Step 4️⃣ : Train all three models and compare
rnn_types = ["SimpleRNN", "LSTM", "GRU"]
history_dict = {}
```

```
for rnn in rnn_types:
    print(f"\nTraining {rnn} model...")
    model = build_rnn(rnn, activation="tanh")
    history = model.fit(x_train, y_train,
                        epochs=4,
                        batch_size=64,
                        validation_split=0.2,
                        verbose=1)
    history_dict[rnn] = history
    print(f"{rnn} Evaluation:")
    model.evaluate(x_test, y_test, verbose=1)
    print("="*60)

# Step 5️⃣ : Compare Accuracy & Loss Graphically
plt.figure(figsize=(12,5))

for rnn in rnn_types:
    plt.plot(history_dict[rnn].history['val_accuracy'], label=f'{rn
plt.title("Validation Accuracy Comparison")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.show()

plt.figure(figsize=(12,5))
for rnn in rnn_types:
    plt.plot(history_dict[rnn].history['val_loss'], label=f'{rnn} V
plt.title("Validation Loss Comparison")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.show()

# Step 6️⃣ : Evaluate final models on test set
results = []
for rnn in rnn_types:
    model = build_rnn(rnn)
    model.fit(x_train, y_train, epochs=2, batch_size=64, verbose=0)
    loss, acc = model.evaluate(x_test, y_test, verbose=0)
    results.append((rnn, acc))

print("\n📊 Final Accuracy Comparison:")
for model_name, acc in results:
    print(f"{model_name}: {acc*100:.2f}%")
```

```
Loading IMDB dataset...
Downloading data from https://storage.googleapis.com/tensorflow/tf-k
17464789/17464789 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Training samples: 25000, Test samples: 25000
Data shape after padding: (25000, 200) (25000, 200)

Training SimpleRNN model...
Epoch 1/4
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedd
  warnings.warn(
313/313 ━━━━━━━━━━━━━━━━━━━━ 34s 100ms/step - accuracy: 0.5374 - los
Epoch 2/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 30s 96ms/step - accuracy: 0.7244 - loss
Epoch 3/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 42s 100ms/step - accuracy: 0.9140 - los
Epoch 4/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 30s 95ms/step - accuracy: 0.9756 - loss
SimpleRNN Evaluation:
782/782 ━━━━━━━━━━━━━━━━━━━━ 13s 17ms/step - accuracy: 0.7658 - loss
============================================================

Training LSTM model...
Epoch 1/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 87s 271ms/step - accuracy: 0.6717 - los
Epoch 2/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 142s 271ms/step - accuracy: 0.9029 - lo
Epoch 3/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 87s 278ms/step - accuracy: 0.9300 - los
Epoch 4/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 137s 263ms/step - accuracy: 0.9503 - lo
LSTM Evaluation:
782/782 ━━━━━━━━━━━━━━━━━━━━ 28s 36ms/step - accuracy: 0.8588 - loss
============================================================

Training GRU model...
Epoch 1/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 92s 285ms/step - accuracy: 0.6676 - los
Epoch 2/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 90s 288ms/step - accuracy: 0.8950 - los
Epoch 3/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 90s 287ms/step - accuracy: 0.9388 - los
Epoch 4/4
313/313 ━━━━━━━━━━━━━━━━━━━━ 90s 286ms/step - accuracy: 0.9612 - los
GRU Evaluation:
782/782 ━━━━━━━━━━━━━━━━━━━━ 27s 34ms/step - accuracy: 0.8553 - loss
============================================================
```




0.775
0.750
0.725
0.700
0.0
0.5
1.0
1.5
2.0
2.5
3.0
Epochs
SimpleRNN Validation Accuracy
LSTM Validation Accuracy
GRU Validation Accuracy


Validation Loss Comparison
Loss
0.65
0.60
0.55
0.50
0.45
0.40
0.35
0.30
0.0
0.5
1.0
1.5
2.0
2.5
3.0
Epochs
SimpleRNN Validation Loss
LSTM Validation Loss
GRU Validation Loss